Eleição de Líder pelo Algoritmo do Valentão e Detecção de Falhas por Heartbeat

Vinicius Forato Coracin (RA 231528)
IC - Unicamp
v231528@dac.unicamp.br
Repositório (GitHub): *https://github.com/ViniciusFCoracin/MC714/tree/main/Lab02*
Vídeo: *https://drive.google.com/file/d/1D5n4JMlY0bT3Pu342bJHDMALZPdM83ZP/view?usp=sharing*

***Resumo*—Foi implementado o algoritmo do Valentão (*Bully*) para eleição de líder integrado a um detector de falhas por heartbeat, em Python, com cada nó executando como processo ou contêiner independente e comunicando-se por TCP. Foi declarado um modelo parcialmente síncrono com falhas *crash-stop* e foram verificadas propriedades de segurança e vivacidade por testes automatizados e pela análise dos logs. No experimento principal, o líder é terminado com SIGKILL. O número de nós foi variado para $N \in \{3, 5, 7, 9\}$, com 10 repetições cada. O tempo de recuperação acompanha o tempo de detecção ($\approx$ o *timeout* do detector) e é pouco sensível a N, enquanto o número de mensagens de eleição cresce de forma aproximadamente quadrática. Percebemos que, sem encerrar antecipadamente a espera por respostas quando o detector suspeita de todos os nós consultados, a recuperação fica bimodal e mais lenta.**

I. Introdução

Muitos serviços distribuídos precisam de um nó responsável por coordenar os outros nós, chamado de *líder*, além de uma maneira de detectar quando ele falha. Neste sentido, este trabalho implementa e avalia dois algoritmos complementares da lista do enunciado: a *eleição de líder pelo algoritmo do Valentão (Bully)*, em que o nó vivo de maior identificador vence, e um *detector de falhas por heartbeat*, que fornece suspeitas à eleição. Esses dois algoritmos são independentes, mas complementares: enquanto o detector dispara a eleição quando suspeita de algum problema com o líder, a eleição é responsável por eleger um novo, dentre aqueles nós que não são detectados como suspeitos. As seções seguintes estão organizadas da seguinte forma: a apresentação o modelo, na Seção II; a arquitetura, na Seção III; os algoritmos, na Seção IV; os testes, na Seção V; a reprodução, na Seção VI; a metodologia, na Seção VII; os resultados, na Seção VIII; e as limitações, na Seção IX.

II. Modelo do Sistema

A. Propriedades de correção

Sem partição, com N nós dos quais um subconjunto falha por *crash-stop*:

- **S1 (acordo):** em quiescência, todos os nós vivos têm o mesmo líder.
- **S2 (validade):** esse líder é o de maior id entre os nós vivos.
- **L1 (terminação):** se o líder falha, todo nó vivo adota um novo líder em tempo finito.

Tabela I
Hipóteses do modelo do sistema

Item	Hipótese e justificativa
Modelo temporal	**Parcialmente síncrono**: existem limites de atraso, mas desconhecidos. Em contêineres num mesmo host os atrasos costumam ser pequenos, porém sujeitos a picos (escalonamento do SO); por isso o algoritmo depende de *timeouts* e aceita suspeitas incorretas.
Canais	TCP ponto a ponto, uma conexão persistente por par, sem duplicação, FIFO por par (exceto com atraso injetado). Perdas ocorrem só quando injetadas ou ao reconectar com um destino fora do ar.
Falhas	*Crash-stop* de qualquer nó (inclusive o líder), via SIGKILL; reinício como nova incarnação (*crash-recovery* simples); partições injetadas na aplicação. Sem falhas bizantinas.
Relógios	O algoritmo usa apenas intervalos medidos localmente; não há sincronização de relógios. Os *timestamps* dos logs vêm do relógio do host, válido para medir durações porque todos os nós rodam na mesma máquina.
Persistência	Nenhuma. Um nó reiniciado começa sem líder e dispara eleição.

- **FD1 (completude):** todo nó que falha é suspeito por todo nó vivo.
- **FD2 (acurácia eventual):** a partir de algum instante nenhum nó vivo é suspeito. Com *timeout* fixo não é garantida em geral; observamos zero falsas suspeitas nos experimentos. O *timeout* adaptativo implementado (`HB_TIMEOUT_INCREMENT`) é a via para obtê-la sob sincronia parcial. Nos experimentos principais, foi usado `HB_TIMEOUT_INCREMENT=0`; portanto, eles não avaliam quantitativamente essa adaptação.

Com partição, S1 e S2 valem apenas dentro de cada lado. Após a cura, um conflito de líderes é resolvido por *heartbeats* (Seç. IV-C).

Essas propriedades devem ser interpretadas em conjunto com as hipóteses do modelo. O Bully não impede dois nós de se declararem líderes durante uma partição, pois cada lado pode observar apenas uma parte do sistema, ou seja, um

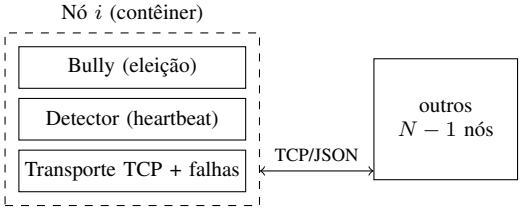


Fig. 1. Arquitetura de um nó: eleição sobre detector sobre transporte com injeção de falhas.

acordo global pode não ocorrer nesse momento. A propriedade relevante é a convergência após a cura: ao receber um heartbeat que anuncia um líder de identificador maior e não suspeito, o nó reativa o processo de eleição. O maior identificador observado então volta a vencer.

Além disso, é importante esclarecer que aquilo que o detector faz é uma dedução/suspeita, e não uma prova da falha. Ele infere uma possível falha a partir do silêncio observado durante T_{to}, por isso, sob atrasos arbitrariamente grandes, um processo correto pode ser considerado suspeito. A janela de aquecimento usada nos experimentos evita medir a subida escalonada dos processos como uma falha, mas não transforma o detector em um detector perfeitamente acurado sob qualquer condição de rede.

III. Arquitetura

Cada nó lógico é um processo (ou contêiner) independente, identificado por um inteiro, e se comunica exclusivamente por rede (Fig. 1). A configuração (id, lista de pares, intervalos, *timeouts*, perda, atraso, semente) vem de variáveis de ambiente, de modo que a mesma imagem serve ao ambiente local e a outros ambientes.

Transporte. Mensagens JSON, uma por linha, sobre TCP, com uma conexão persistente por par e *timeouts* explícitos de conexão e escrita. A primeira versão abria uma conexão por mensagem; no Windows isso esgotou portas efêmeras (erro `WinError 10048`) em execuções longas, o que motivou a mudança. A camada também injeta falhas de forma repetível por semente: perda, atraso e partição. **Observabilidade.** Cada nó registra eventos JSON (`suspect`, `trust`, `election_start`, `early_abort`, `elected`, `leader`, `stats`); as propriedades e as métricas são calculadas a partir desses logs.

O ciclo de vida de um nó é composto por quatro atividades concorrentes. A primeira envia periodicamente os *heartbeats* para todos os pares. A segunda executa as verificações do detector e dispara uma eleição quando a suspeita corresponde ao líder conhecido. A terceira trata as mensagens de eleição e coordenação. A quarta registra contadores cumulativos de mensagens, usados posteriormente para medir o custo da eleição sem depender apenas da saída textual dos processos. Essa decomposição mantém o protocolo distribuído independente da forma de execução: no modo local, cada nó é um processo Python; no modo de contêiner, cada processo é encapsulado em uma imagem Docker.

O transporte é deliberadamente simples. Cada mensagem contém um tipo, o identificador de origem e os campos específicos do protocolo, sendo serializada como uma linha JSON. As conexões de saída são mantidas por par e protegidas por uma trava que serializa reconexões e escritas. Quando uma conexão falha, a camada não bloqueia indefinidamente: ela aplica um tempo limite explícito e um pequeno "backoff", deixando que o detector decida quando o destino deve ser suspeito. Essa decisão é importante no Windows, onde abrir um novo socket para cada heartbeat produz muitos sockets em estado `TIME_WAIT`.

As falhas de comunicação são injetadas na própria camada de transporte. A perda descarta uma mensagem antes do envio; o atraso introduz uma espera aleatória; e a partição faz o transporte ignorar determinados identificadores. Como esses mecanismos usam sementes por nó, os cenários podem ser repetidos sem alterar o algoritmo principal. A mesma camada também recebe comandos de controle para bloquear ou desbloquear pares durante uma demonstração de partição.

IV. Algoritmos

A. Detector de falhas por heartbeat

Cada nó envia `HB` a todos a cada T_{hb} (carregando o líder em que acredita) e suspeita de um par que não ouve há mais que T_{to}. Se um suspeito volta a ser ouvido, é re-confiado e, opcionalmente, seu *timeout* cresce. Se o último `HB` chegou $u \in [0, T_{hb}]$ antes da falha e a verificação ocorre a cada $T_{hb}/2$, o tempo de detecção fica em $(T_{to} - T_{hb},\ T_{to} + T_{hb}/2]$ após a falha (mais o atraso de rede).

B. Algoritmo do Valentão

Mensagens: `ELECTION` (a ids maiores), `ANSWER` (resposta de um maior) e `COORDINATOR` (anúncio do vencedor). Regras:

1) Um nó inicia eleição ao subir ou ao suspeitar do líder.
2) Envia `ELECTION` aos ids maiores. Se nenhum responde, torna-se líder e envia `COORDINATOR` a todos; se recebe `ANSWER`, espera um `COORDINATOR` e, se estourar o tempo, recomeça.
3) Ao receber `ELECTION` de um id menor, responde `ANSWER` e inicia a própria eleição.
4) Ao receber `COORDINATOR` de um id maior, adota-o; de um id menor, reafirma-se iniciando eleição.

C. Integração e extensões ao algoritmo clássico

Algumas adaptações no algoritmo clássico do Bully foram realizadas:

- **Eleição ciente do detector:** nós suspeitos não são consultados; se todos os maiores são suspeitos, o nó assume imediatamente a liderança.
- **Encerramento antecipado da espera** (`FD_EARLY_ABORT`): se, durante a espera por `ANSWER`, o detector passa a suspeitar de todos os nós consultados, a espera termina. Sem isso, um nó consultado alguns milissegundos antes de detectar a queda consulta o líder morto e espera um *timeout* de eleição inteiro.

- **Resolução de conflito:** se um `HB` informa um líder maior que o meu e não suspeito, inicio eleição. Isso reconcilia os líderes após uma partição curar.

Na inicialização, o detector aguarda uma janela de tolerância antes de produzir a primeira suspeita. Essa janela, configurada por `FD_STARTUP_GRACE`, tem como valor padrão `STARTUP_DELAY + HB_TIMEOUT`. Sem ela, um processo que subisse alguns instantes depois dos demais poderia ser classificado como falho antes de enviar seu primeiro heartbeat. Depois desse período, o detector mantém, para cada par, o instante do último heartbeat e um estado de suspeita. O recebimento de uma nova mensagem remove a suspeita e pode aumentar o timeout daquele par quando a adaptação está habilitada.

Uma eleição é executada como uma tarefa única por nó; novas solicitações enquanto já existe uma eleição em andamento não criam tarefas concorrentes. Isso evita que várias suspeitas do mesmo líder produzam uma explosão desnecessária de eleições. Durante uma eleição, o nó consulta somente identificadores maiores que o seu e, quando a opção correspondente está habilitada, elimina da consulta os pares já suspeitos. Se algum nó maior responder, o candidato aguarda o anúncio de coordenação. Se nenhum responder, ele se declara líder e anuncia essa decisão aos demais. O evento de liderança só é registrado quando o valor realmente muda, o que facilita a extração dos estados finais nos testes.

D. Complexidade esperada

Os *heartbeats* custam $N(N-1)$ mensagens por período, ou seja, tráfego de fundo $N(N-1)/T_{hb}$ (720 msg/s para $N = 9$ e $T_{hb} = 0{,}1$ s). Uma eleição envia no pior caso $\sum_k (N-k) = N(N-1)/2$ `ELECTION`, até o mesmo número de `ANSWER` e $N-1$ `COORDINATOR`: $O(N^2)$. O tempo de recuperação esperado é $\approx T_{to}$ mais o custo da eleição, que com o encerramento antecipado é de poucos atrasos de rede.

V. Testes de Correção

Oito testes automatizados executam nós reais em localhost por TCP e o *crash-stop* é feito derrubando o nó sem aviso. Os testes abrangem: a eleição inicial (maior id vence; sem falsas suspeitas); o crash do líder (reeleição, completude e convergência < 2 s); o crash do não-líder (líder inalterado); o retorno do líder (retoma a liderança); 10% de perda de mensagens (converge); partição e cura; recuperação independente de *timeout* de eleição longo; e o detector isolado com relógio simulado. As propriedades S1, S2, L1, FD1 e FD2 são verificadas pelos mesmos logs em `log_checks.py`. Notamos que a completude (FD1) é eventual, pois nosso primeiro teste a afirmava de imediato e falhava eventualmente, pois outros nós adotam o novo líder antes do próprio detector disparar. Na validação final todos os oito testes passaram.

O teste de eleição inicial verifica simultaneamente acordo e validade: depois da estabilização, todos os nós devem registrar o maior identificador como líder. Os testes de crash separam duas situações que têm consequências diferentes para o protocolo. Ao derrubar o líder, todos os nós vivos devem convergir para o maior identificador restante; ao derrubar um não-líder, o líder atual deve permanecer. O teste de reinício verifica que uma nova incarnação não reutiliza estado antigo e, ainda assim, recupera a liderança quando possui o maior identificador.

Os cenários de perda e partição exercitam a integração, e não apenas funções isoladas. Com 10% de perda, a eleição deve continuar convergindo porque mensagens podem ser retransmitidas implicitamente por novas rodadas do protocolo e pelo detector. Na partição, cada lado pode eleger seu maior nó; após a liberação dos bloqueios, os heartbeats carregam a informação de um líder maior e provocam a reconciliação. O detector isolado usa um relógio simulado para testar precisamente o limiar de timeout, a remoção de uma suspeita e o crescimento adaptativo do timeout.

Os verificadores em `log_checks.py` não dependem de uma ordem fixa de linhas. Eles ordenam os eventos pelos timestamps e extraem o último líder observado por cada nó vivo. Essa escolha permite que os testes tolerem a concorrência natural entre detecção, eleição e entrega de mensagens, verificando propriedades de estado final e tempos de convergência em vez de exigir uma sequência artificial de eventos.

VI. Reprodução e Execução

O repositório contém o código, os testes, o arquivo `compose.yaml`, os scripts de experimento e os resultados. Em uma instalação Python 3.10 ou superior, os testes e o experimento local podem ser executados com:

```
pip install -r requirements.txt
python -m pytest -v tests/
python experiments/run_experiments.py \
  --nodes 3 5 7 9 --reps 10 --out results
python experiments/plot_results.py --in results
```

O experimento grava `raw.csv`, `summary.csv` e `env.json`; os dois últimos comandos também geram os gráficos usados neste relatório. O arquivo `compose.yaml` é a configuração padrão com cinco nós. Para executar o mesmo ambiente com outro número de nós, geramos os também os arquivos `compose.n3.yaml`, `compose.n7.yaml` e `compose.n9.yaml`. Para rodar, basta executar:

```
docker compose up --build -d
docker compose logs -f
docker kill node5
docker start node5
docker compose down -v
```

Em uma demonstração com cinco nós, espera-se inicialmente o nó 5 como líder. O comando `docker kill node5` deve provocar suspeitas e a eleição do nó 4; depois de `docker start node5`, o nó 5 deve recuperar a liderança por ter o maior identificador. Para outros valores de N, o maior id é o líder esperado.

VII. Metodologia Experimental

Cenário. Sobem-se N processos locais ou N contêineres gerados pelo `generate_compose.py`; após todos concordarem no líder N e 1 s de estado estacionário, o líder é terminado com SIGKILL e mede-se a recuperação. **Parâmetro**

variado: $N \in \{3, 5, 7, 9\}$; 10 repetições por nível com sementes 1000–1009; $T_{hb} = 0{,}1$ s, $T_{to} = 0{,}5$ s, *timeout* de eleição 0,3 s, sem perda nem atraso. Como não há perda, a semente não altera a execução; a variação entre repetições vem do escalonamento do SO e da fase dos *heartbeats*. **Métricas:** (i) detecção: tempo até o primeiro e até o último nó vivo suspeitar do líder; (ii) convergência: tempo até todos os vivos adotarem o novo líder; (iii) mensagens de eleição (`ELECTION`+`ANSWER`+`COORDINATOR`) após a falha, sem contar *heartbeats*; (iv) falsas suspeitas. Latências são reportadas com média, desvio, p50 e p95. A variação de N é relevante para ambos os componentes: para o detector, permite observar detecção e falsas suspeitas sob diferentes quantidades de pares; para o Bully, permite medir o custo da eleição. Os testes separados também cobrem perda de mensagens e partição.

Cada rodada é considerada válida somente depois que todos os nós concordam no líder inicial. Em seguida, aguarda-se uma janela de estado estacionário para que os contadores de mensagens e os estados do detector não reflitam a inicialização. O instante t_0 é registrado imediatamente antes do crash do líder. A detecção é calculada pela diferença entre o primeiro ou o último evento `suspect` relativo ao líder e t_0; a convergência é o instante em que o último nó vivo registra o novo líder. Rodadas que não convergem não entram nas estatísticas de latência, mas permanecem no arquivo bruto com seu estado e motivo, preservando a possibilidade de auditar a taxa de sucesso.

As mensagens de eleição são obtidas por diferenças entre contadores cumulativos dos eventos `stats`. Assim, heartbeats e tentativas de conexão não são confundidos com o custo do Bully. Para FD2, suspeitas de nós vivos são contadas somente depois do aquecimento; suspeitas do nó deliberadamente terminado não são classificadas como falsos positivos. Essa separação é necessária porque o início simultâneo dos processos não é garantido pelo sistema operacional.

Tabela II
AMBIENTE E CONFIGURAÇÃO

Item	Valor
Hardware	4 processadores lógicos; Intel64 Family 6 Model 126
Sistema operacional	Windows 11
Python / Docker / Compose	Python 3.12.9; Docker
Execução dos nós	processos locais independentes, comunicando-se por TCP
Duração do ensaio	aproximadamente 2 min 25 s para 40 rodadas
Aquecimento	acordo inicial + 1 s de estado estacionário; janela FD inicial de 0,8 s

VIII. Resultados e Discussão

Tempo de recuperação. A convergência (≈0,49–0,52 s) acompanha a detecção e coincide com a previsão $(T_{to} - T_{hb}, T_{to} + T_{hb}/2]$ da Seç. IV-A, sem tendência clara em N no intervalo testado (p95 $\leq 0{,}550$ s). Não houve falsas suspeitas nas 40 execuções após o aquecimento ($T_{to} = 5T_{hb}$, sem perda). **Custo em mensagens.** O número médio cresce de 5,6 para 105,8 quando N triplica, compatível com o crescimento quadrático esperado. O desvio aumenta em $N = 9$: o número de nós que iniciam eleição antes de receber o `COORDINATOR` varia com a fase dos detectores e o escalonamento do SO.

Tabela III
RECUPERAÇÃO APÓS O CRASH DO LÍDER (10 REPETIÇÕES; TEMPOS EM S)

N	Detecção (último)	Convergência	p50 / p95	Msgs. eleição
3	$0{,}493 \pm 0{,}037$	$0{,}491 \pm 0{,}038$	0,497 / 0,532	$5{,}6 \pm 1{,}3$
5	$0{,}502 \pm 0{,}036$	$0{,}498 \pm 0{,}038$	0,492 / 0,544	$24{,}8 \pm 8{,}5$
7	$0{,}491 \pm 0{,}026$	$0{,}491 \pm 0{,}026$	0,490 / 0,529	$58{,}3 \pm 19{,}3$
9	$0{,}512 \pm 0{,}030$	$0{,}515 \pm 0{,}030$	0,516 / 0,550	$105{,}8 \pm 64{,}3$

O comportamento temporal é coerente com o papel dos dois algoritmos. O detector é o primeiro componente a reagir ao crash, pois o Bully não possui informação independente de falha; ele só inicia uma nova eleição quando recebe a suspeita. Como $T_{to} = 0{,}5$ s e $T_{hb} = 0{,}1$ s, a maior parte das observações fica próxima de meio segundo, com pequena dispersão causada pela posição do último heartbeat no instante da falha e pelo período de verificação. Depois da primeira suspeita, a eleição é curta no cenário testado, pois os nós que já suspeitam do líder não aguardam respostas dele.

O aumento do custo de mensagens é mais irregular que o tempo de recuperação. Em uma execução ideal, somente os nós necessários participariam da eleição e o custo seria próximo da curva quadrática de referência. Na implementação distribuída, entretanto, vários nós podem detectar a falha em instantes diferentes. Um nó que ainda não suspeitou do líder pode iniciar uma rodada adicional, receber um `ANSWER` e aguardar o `COORDINATOR`; por isso algumas sementes produzem mais mensagens que outras. Esse efeito explica tanto o desvio de $N = 9$ quanto os valores acima da referência quadrática no gráfico, sem contradizer a complexidade assintótica $O(N^2)$.

A ausência de falsas suspeitas nas 40 rodadas deve ser lida com a mesma ressalva. Ela foi observada sem perda nem atraso injetados, depois da janela de aquecimento e com $T_{to} = 5T_{hb}$. Não é uma garantia de acurácia para uma rede arbitrariamente lenta. Nos testes funcionais, a perda de mensagens, a partição e a cura são avaliadas separadamente para demonstrar robustez do protocolo, enquanto o experimento principal isola o efeito do número de nós sobre recuperação e custo da eleição.

Ablação do encerramento antecipado. A implementação oferece a opção `--no-early-abort`, que permite comparar a espera tradicional por `ANSWER` com a versão integrada ao detector. A medição quantitativa específica dessa ablação fica como extensão, pois os resultados principais usam a configuração padrão com `FD_EARLY_ABORT=1`.

IX. Limitações do experimento

- Contêineres numa mesma máquina *não* são domínios de falha independentes: um SIGKILL a um nó não exerce

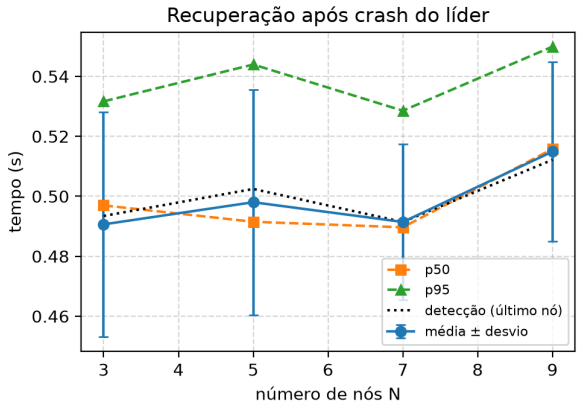


Fig. 2. Detecção e convergência após o crash do líder em função de N.

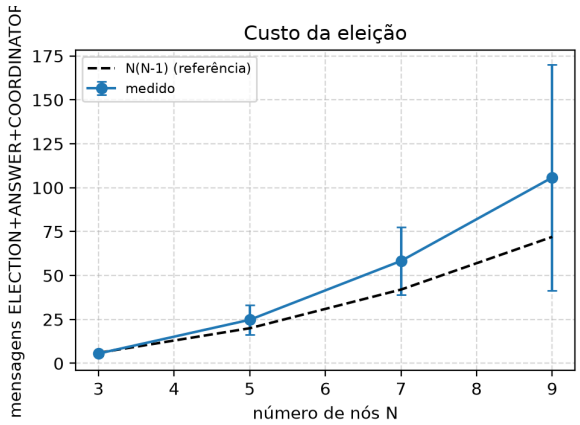


Fig. 3. Mensagens de eleição em função de N, com referência $N(N-1)$.

falhas correlacionadas de rede, disco ou máquina. Todos os nós compartilham CPU, o que pode atrasar *heartbeats* sob carga.

- Relógio único: as durações são confiáveis neste ambiente, mas não seriam em máquinas distintas sem sincronização.
- Só *crash-stop*, sem persistência e sem falhas bizantinas; $N \leq 9$.
- *Timeout* fixo nos experimentos: a acurácia observada vale para a configuração testada (sem perda), não em geral.
- Perda de mensagens e partição são cobertas pelos testes automatizados, mas não fazem parte das medições do experimento principal.

X. Conclusões

O Bully integrado a um detector por *heartbeat* recupera-se da queda do líder em tempo dominado pelo *timeout* de detecção, com custo em mensagens $O(N^2)$ e alta variabilidade. O principal aprendizado de projeto foi que detalhes de integração (consultar ou não nós suspeitos, e encerrar a espera quando todos viram suspeitos) mudam qualitativamente a latência de recuperação, o que só ficou visível pelos logs e pela ablação. Como implementação futura, poderíamos variar o detector (*timeout*, perda), usar *timeout* adaptativo, executar na nuvem do IC, e medir partições e falhas combinadas.

Código Reutilizado

Os nós usam apenas a biblioteca padrão do Python (`asyncio`); `pytest`, `pandas` e `matplotlib` são usados em testes e análise. O esqueleto inicial do código, dos testes e dos *scripts* foi gerado com auxílio de assistente de IA.

Divisão de Trabalho

O trabalho foi realizado de modo individual por um único aluno. Foi utilizado ferramenta de IA como apoio na geração do código, mais especificamente, na criação de um template para apoiar o início do desenvolvimento e na depuração. Além disso, foi utilizada para realizar a revisão gramatical do texto.

Referências

[1] Prof. Carlos A. Astudillo, Slides de Aula, MC714 2S/2026.
[2] M. van Steen and A. S. Tanenbaum, *Distributed Systems*, 4th ed., distributed-systems.net, 2023.